SYSTEM DESIGN · AGENTIC RAG PORTFOLIO PROJECT

Home Fund

An agent that works out how much cash a specific home takes, projects when you'll get there at your current pace, and shows which spending cuts close the gap. Every model runs on hardware you control.

WHO

Renters saving for a first home who want a real number and a date, not a rule of thumb.

WHAT

Cash needed for a chosen home, a savings chart against your move-in date, and ranked cuts to hit it.

WHY

Budget apps show where money went, not what to change for a specific house. And this data is too sensitive for hosted LLMs.

HOW

Tools do every calculation. A LangGraph agent plans, MCP servers fetch data, RAG cites home-buying guides, and only open-weight models run.

STACK

React + Recharts · FastAPI · LangChain · LangGraph · MCP (FastMCP) · Postgres + pgvector · vLLM · Ollama · Docker

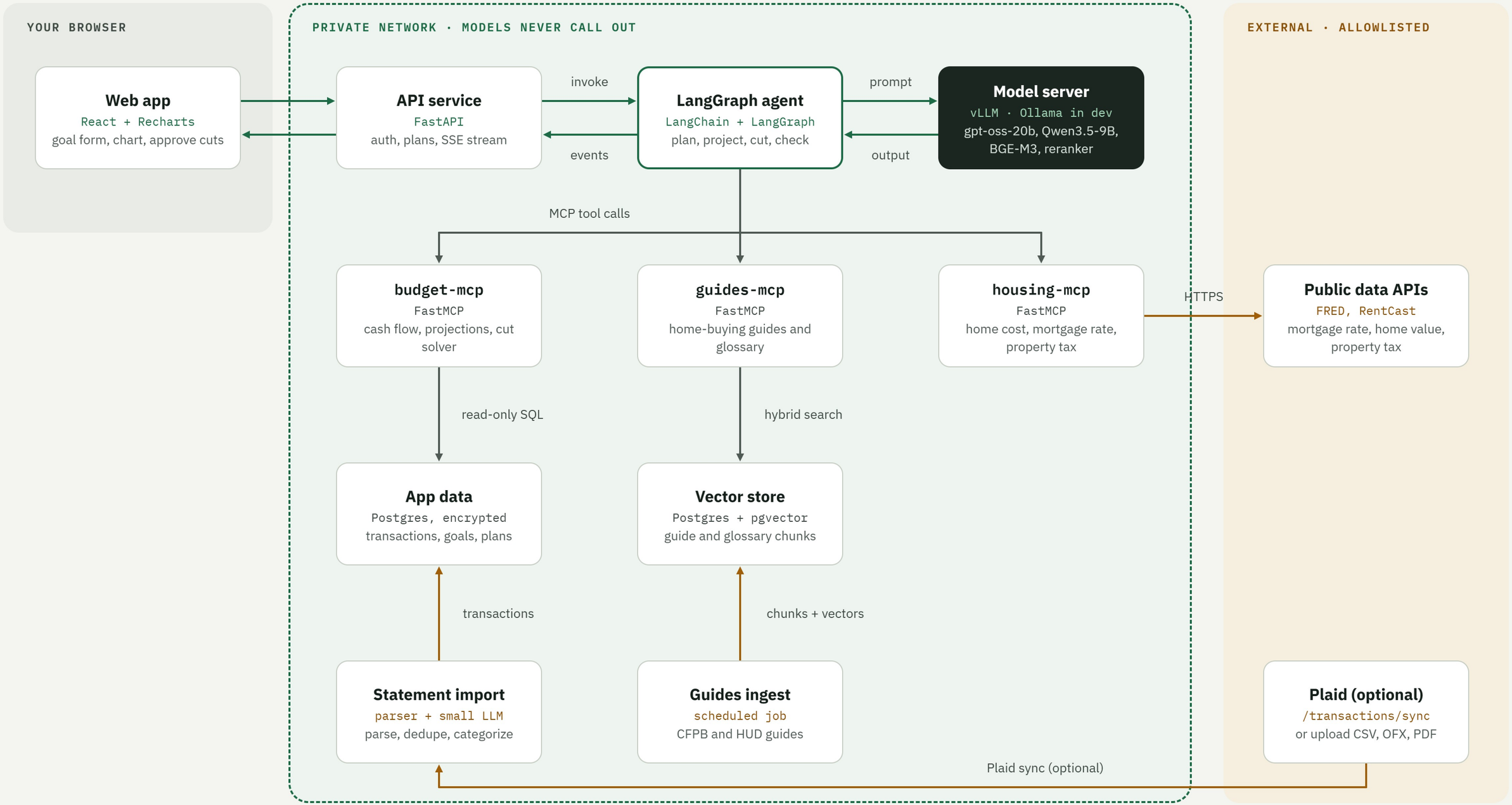
ON THIS CANVAS

01 System architecture
02 Models & security
03 Data & chunking
04 Agent graph
05 Request walkthrough
06 APIs, MCP servers & data
07 App screen

System architecture

request path · tool and data calls · data coming in

Every model runs inside the private network. Only the MCP servers reach the internet, and only allowlisted APIs.



DATA & CHUNKING

Numbers go in tables. Words go in vectors.

Spending math never runs on embedded text. Transactions are rows the tools compute over; only guidance text is chunked for retrieval.

DATA THE AGENT NEEDS

From you

- Take-home pay per month
- Savings already set aside for the home
- 6–12 months of transactions, from Plaid or CSV, OFX and PDF statements
- Fixed bills and minimum debt payments
- Target home price or address, move-in date, down payment (3, 3.5, 10 or 20%)
- ZIP code, for property tax and insurance estimates
- Credit score range, optional; it shifts the rate estimate

From public APIs

- 30-year mortgage rate: FRED series `MORTGAGE30US`, weekly
- Home value estimate and property tax: RentCast
- Insurance and PMI: estimated as a percentage unless you enter a quote

From documents (RAG corpus)

- CFPB home-buying guides and closing-cost explainers
- HUD and FHA basics, such as the 3.5% minimum down payment
- Glossary: PMI, escrow, DTI, closing costs, earnest money

CHUNKING STRATEGY BY SOURCE

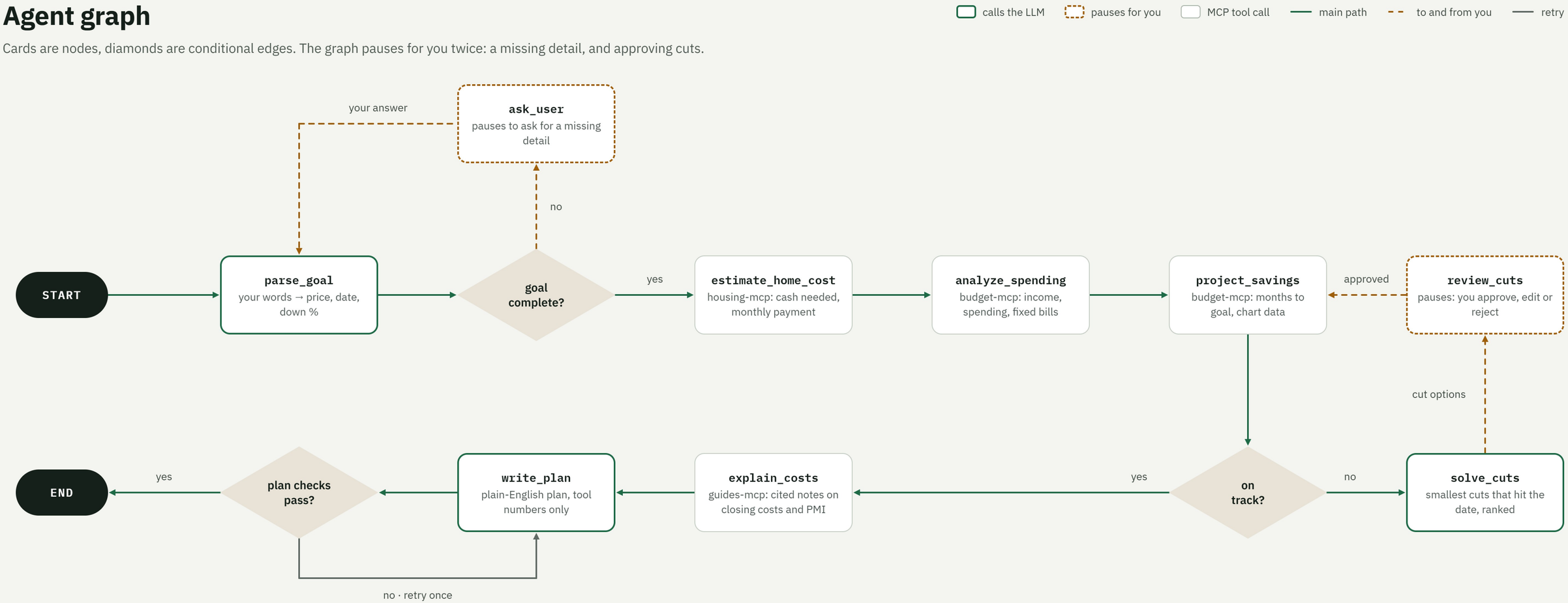
SOURCE	STRATEGY	WHY
Transactions `Plaid JSON, CSV, OFX`	**Don't chunk.** One row per transaction in Postgres. The small model fills in the category; SQL does all the math.	Embedding numbers blurs them. SQL sums them exactly.
Monthly summaries `derived from rows`	**Aggregate, then write.** One short record per category per month, e.g. “Dining · Mar 2026 · $612 · 23 purchases”, handed to the planner by budget-mcp.	The agent explains patterns in words without reading 2,000 rows.
Statement PDFs `bank and card`	**Table-aware parsing.** Extract the transaction table row by row with pdfplumber or Docling, never splitting a row. The account summary block stays whole.	A row split across chunks turns into two wrong transactions.
CFPB and HUD guides `long prose`	**Heading-aware split, parent-child.** About 500-token chunks with 50 overlap. Prepend “document › section” before embedding, and hand the model the parent section.	Small chunks match precisely; the parent section gives full context.
Glossary `PMI, escrow, DTI`	**One term per chunk.** Small enough to pass whole.	“What is PMI?” is answered in a single hit.

RETRIEVAL RECIPE

Question › BGE-M3 dense + sparse search › Filter by source and tax year › Top 20 › bge-reranker-v2-m3 › **Top 5, parent sections into the prompt**

Agent graph

Cards are nodes, diamonds are conditional edges. The graph pauses for you twice: a missing detail, and approving cuts.



GRAPH STATE

```
class PlanState(TypedDict):
    goal: HomeGoal              # price, zip, move_in, down_pct
    home_cost: CostBreakdown    # down payment, closing, reserves, PITI
    cashflow: Cashflow          # income, spend by category, fixed bills
    projection: list[Point]     # month, balance, target
    cuts: list[Cut]             # category, amount, ease, approved
    notes: list[Citation]       # cited explanations of each cost
    plan: str
    review_rounds: int          # max 3
```

GUARDS

Math lives in tools

Every dollar figure comes from budget-mcp or housing-mcp. Checks recompute each one before the plan is shown.

You approve first

interrupt() pauses at review_cuts, and a Postgres checkpointer resumes the graph when you answer.

Essentials are protected

Rent, insurance, utilities and minimum debt payments can't be cut unless you unlock them.

Loops are bounded

Up to 3 review rounds and 1 plan retry, with a recursion limit as a backstop.

REQUEST WALKTHROUGH

One goal, end to end

“I want a $420,000 home with 10% down by June 2028.” Illustrative numbers, traced through every node.

1 **Set the goal** `Web app`

You enter a $420,000 home, your ZIP, 10% down and a June 2028 move-in. Six months of statements were imported earlier.

2 **parse_goal** `Qwen3.5-9B`

Structured output fills a HomeGoal. Nothing is missing, so ask_user is skipped.

3 **estimate_home_cost** `housing-mcp`

Pulls the 30-year rate from FRED and property tax from RentCast. Cash needed: $42,000 down + $12,600 closing + $6,300 reserves = $60,900.

4 **analyze_spending** `budget-mcp`

Six-month averages: $6,800 take-home, $5,600 spent, $1,200 saved a month. $18,000 is already set aside.

5 **project_savings** `budget-mcp`

A $42,900 gap at $1,200 a month takes 36 months: October 2029, 16 months late. on track? says no.

6 **solve_cuts** `budget-mcp + gpt-oss-20b`

June 2028 needs about $2,145 a month. The solver finds $952 across five flexible categories; rent and minimum payments stay untouched.

7 **review_cuts** `you`

The graph pauses. You approve the cuts, it resumes and re-projects: $2,152 a month reaches the goal in 20 months.

8 **explain_costs** `guides-mcp`

Retrieves cited CFPB guidance on closing costs, reserves and PMI, so the plan can explain each line of the $60,900.

9 **write_plan + checks** `gpt-oss-20b`

The plan is written from tool numbers only. Checks recompute every figure, then the plan and chart are saved.

WHAT YOU SEE

Cash needed $42,000 down · $12,600 closing · $6,300 reserves	**$60,900**
At your current pace $1,200 a month	**Oct 2029**
With approved cuts $2,152 a month	**Jun 2028**

Dining and delivery	−$300
Travel fund	−$280
Shopping	−$180
Rideshare	−$120
Subscriptions, 7 down to 3	−$72
Total per month	**−$952**

Protected: rent, insurance, utilities and minimum debt payments.

Illustrative numbers. A planning tool, not financial advice.

APIS, MCP SERVERS & DATA MODEL

Three MCP servers, three outside APIs, one database

MCP SERVERS · BUILT WITH FASTMCP, INSIDE THE PRIVATE NETWORK

budget-mcp

`get_cashflow(months=6)`
Take-home, spending and amount saved

`get_spending_by_category(months=6)`
Monthly summary records per category

`get_fixed_bills()`
Rent, insurance, minimum payments

`project_savings(start, monthly, target)`
Balance by month for the chart

`solve_cuts(gap_per_month, protected)`
Smallest set of cuts, capped per category

Reads Postgres with a read-only role

housing-mcp

`estimate_home_cost(price, zip, down_pct)`
Down payment, closing, reserves, PITI, PMI

`get_mortgage_rate()`
FRED MORTGAGE30US, cached weekly

`get_property(address)`
Value estimate and property tax from RentCast

Egress only to FRED and RentCast

guides-mcp

`search_guides(query, k=5)`
Hybrid search, reranked, with citations

`define_term(term)`
Glossary entry for PMI, escrow, DTI and more

`get_section(chunk_id)`
Parent section for full context

Reads pgvector; embeds and reranks on the model server

Plaid's own MCP servers cover developer diagnostics, not a user's transactions, so budget-mcp reads transactions you've already synced or uploaded.

OUTSIDE APIS

Plaid `/transactions/sync`
Optional. Skip it and upload CSV, OFX or PDF statements instead.

FRED `MORTGAGE30US`
Weekly 30-year fixed average. Free API key.

RentCast `value estimate, property records`
Free tier covers 50 requests a month, enough for a demo.

APP API · FASTAPI

`POST` **`/api/imports`**
Upload statements; parsed and categorized in the background.

`POST` **`/api/plans`**
Goal in; streams progress, chart data and cuts over SSE.

`POST` **`/api/plans/:id/review`**
Approve or edit cuts; resumes the paused graph.

`GET` **`/api/plans/:id`**
Saved plan, projection and citations.

POSTGRES TABLES

transactions

id	uuid PK
user_id	uuid FK
posted_on	date
amount	numeric(12,2)
merchant	text
category	text
is_fixed	boolean
source	text

goals

id	uuid PK
user_id	uuid FK
price	numeric
zip	text
down_pct	numeric
move_in	date
saved	numeric

plans

id	uuid PK
goal_id	uuid FK
status	text
cash_needed	numeric
monthly_target	numeric
projection	jsonb
cuts	jsonb
citations	jsonb

guide_chunks

id	uuid PK
parent_id	uuid
content	text
dense	vector(1024)
sparse	sparsevec
metadata	jsonb

HNSW index on dense; source, section and tax year live in metadata

LangGraph's Postgres checkpointer adds its own tables. That's what lets the graph pause at review_cuts and pick up where it left off.